Day 93 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 14, Day 3 reference

Watch: Temperature, Top-K & Top-P Explained in 10 Minutes (bababoss) | Nucleus Sampling: The Curious Case of Neural Text Degeneration (TechViz)

`day93_sampling_strategies.py`, segment by segment, in the order `main()` uses things. Days 90 and 91 are imported and explained there. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: z = the scores (logits) · t = temperature · k = how many tokens top-k keeps · p = the mass top-p keeps

Segment 0 — Setup

Settings: the seven strategies

CODE

```
MODEL = os.path.join(HERE, "day92_mini_gpt.npz")
N_PROMPTS, PROMPT_LEN, GEN_LEN = 20, 32, 400
STRATEGIES = [("greedy", dict(temperature=0)), ("temperature 0.5", dict(temperature=0.5)),
              ("temperature 0.8", dict(temperature=0.8)), ("temperature 1.0",
                  dict(temperature=1.0)),
              ("temperature 1.2", dict(temperature=1.2)), ("top-k 5", dict(top_k=5)), ("top-p
                  0.9", dict(top_p=0.9))]
JUDGE_ORDER = 7
```

- `STRATEGIES = [("greedy", dict(temperature=0)), ...]` — a list of (name, settings) pairs; the settings dict is passed on with **.
- `JUDGE_ORDER = 7` — Day 91's best Kneser-Ney order (6 characters of context), used as the outside judge.

Segment 1 — The sampler

sample_next: four strategies in one function

MATH

temperature: $q \propto exp(z / t)$; top-k: $q \propto p$ on the k largest; top-p: $q \propto p$ on the smallest set with $\Sigma\, p \geq p_0$

CODE

```
def sample_next(logits, rng, temperature=1.0, top_k=None, top_p=None):
    if temperature == 0:
        return logits.argmax(axis=-1)
    z = logits / temperature
    p = np.exp(z - z.max(axis=-1, keepdims=True)); p /= p.sum(axis=-1, keepdims=True)
    if top_k is not None:
        kth = np.sort(p, axis=-1)[:, -top_k][:, None]
        p = np.where(p >= kth, p, 0.0)
    if top_p is not None:
        order = np.argsort(-p, axis=-1)
        sp = np.take_along_axis(p, order, axis=-1)
        keep_sorted = np.cumsum(sp, axis=-1) - sp < top_p
        keep = np.zeros_like(p, dtype=bool); np.put_along_axis(keep, order, keep_sorted, axis=-1)
        p = np.where(keep, p, 0.0)
    p /= p.sum(axis=-1, keepdims=True)
    u = rng.rand(len(p), 1)
    return np.minimum((np.cumsum(p, axis=-1) < u).sum(axis=-1), p.shape[-1] - 1)
```

- `np.sort(p, axis=-1)[:, -top_k][:, None]` — each row's k-th largest probability, as a column, to compare with every entry.
- `np.cumsum(sp, axis=-1) - sp < top_p` — the mass before each sorted token: keep it while that mass is still below p, so the token that crosses p is kept.
- `np.put_along_axis(keep, order, keep_sorted, axis=-1)` — writes the sorted decisions back to the tokens' own positions.
- `(np.cumsum(p, axis=-1) < u).sum(axis=-1)` — inverse-CDF sampling: how many cumulative values are below a uniform number = the drawn token.

Segment 2 — Generation

load_model: Day 92's file back into nested dictionaries

CODE

```
def load_model(path=MODEL):
    f = np.load(path)
    P, cfg = {}, {}
    for k in f.files:
        if k == "vocab":
            continue
        if k.startswith("cfg__"):
            cfg[k[5:]] = int(f[k]); continue
        parts = k.split("__"); node = P
        for part in parts[:-1]:
            node = node.setdefault(part, {})
        node[parts[-1]] = f[k].astype(np.float64)
    return P, [str(v) for v in f["vocab"]], cfg
```

- `k.split("__")` — 'blocks__0__Wq' -> ['blocks', '0', 'Wq'].
- `node = node.setdefault(part, {})` — walks down, creating each nested dictionary the first time.
- `f[k].astype(np.float64)` — back from float32 storage to float64 arithmetic.

generate: all prompts at once

CODE

```
def generate(P, cfg, prompts, n, rng, **how):
    ids = [list(p) for p in prompts]
    for _ in range(n):
        x = np.array([s[-cfg["T"]:] for s in ids])
        lg = d90.gpt_forward(x, P, cfg["h"])[0][:, -1]
        for s, t in zip(ids, sample_next(lg, rng, **how)):
            s.append(int(t))
    return [s[len(p):] for s, p in zip(ids, prompts)]
```

- `**how` — the strategy's settings arrive as keyword arguments and are passed straight on to sample_next.
- `x = np.array([s[-cfg["T"]:] for s in ids])` — the last T tokens of every prompt, stacked into one batch.
- `d90.gpt_forward(x, P, cfg["h"])[0][:, -1]` — only the last position's scores are needed for the next token.

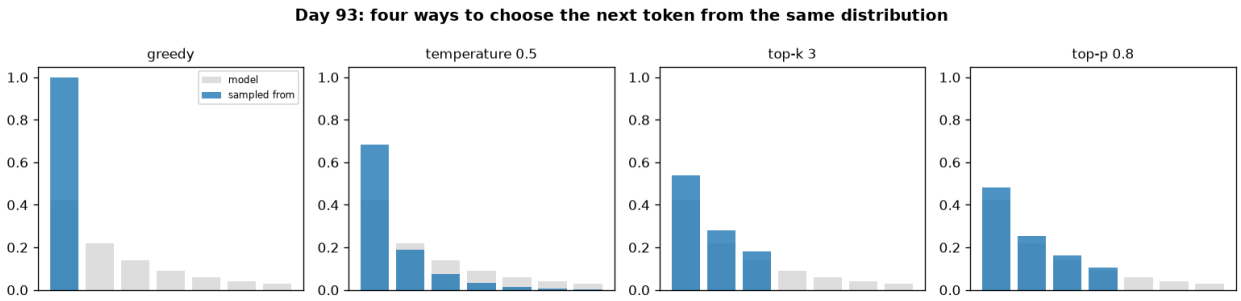
model_bits and text_stats: the measures

CODE

```
def model_bits(P, cfg, prompts, conts):
    full = np.array([list(p) + list(c) for p, c in zip(prompts, conts)])
    tot, cnt = 0.0, 0
    for s in range(PROMPT_LEN, full.shape[1]):
        x = full[:, max(0, s - cfg["T"]):s]
        lp = d90.log_softmax(d90.gpt_forward(x, P, cfg["h"])[0][:, -1])
        tot -= lp[np.arange(len(full)), full[:, s]].sum() / np.log(2); cnt += len(full)
    return tot / cnt
def text_stats(texts, known, judge):
    words = [re.findall(r"[a-z']+", t.lower()) for t in texts]
    pairs = [tuple(w[i:i + 2]) for w in words for i in range(len(w) - 1)]
    flat = [x for w in words for x in w]
    jb = np.mean([judge.bits(t, JUDGE_ORDER) for t in texts if len(t) > JUDGE_ORDER])
    return len(set(pairs)) / max(1, len(pairs)), np.mean([x in known for x in flat]), jb
```

- `re.findall(r"[a-z']+", t.lower())` — the words: runs of letters and apostrophes.
- `tuple(w[i:i + 2])` — a word pair as a tuple, so it can go into a set.
- `judge.bits(t, JUDGE_ORDER)` — Day 91's Kneser-Ney scores the text, skipping its first characters (no context yet).

SYSTEM DIAGRAM



The same distribution cut four ways.

Segment 3 — Experiments

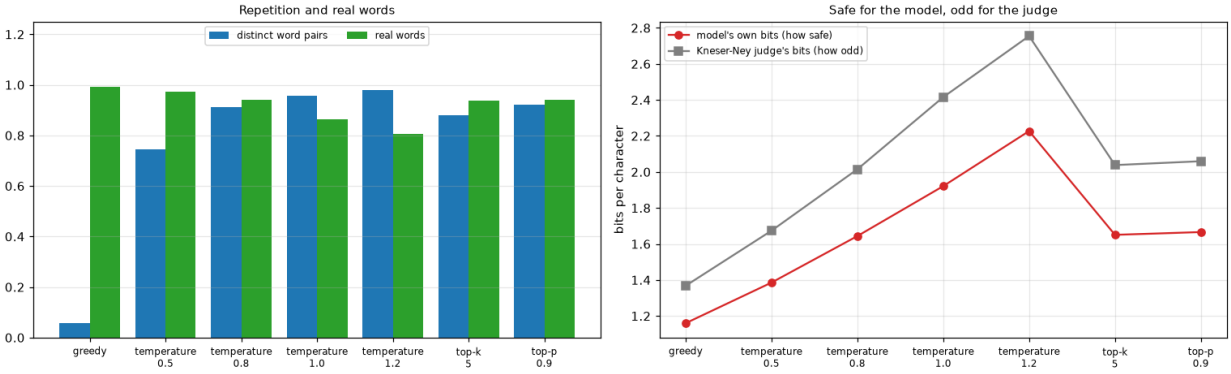
demo_toy: checking the sampler

CODE

```
p = np.array([0.5, 0.2, 0.15, 0.1, 0.05]); lg = np.log(p)[None].repeat(200000, axis=0)
expect = {"temperature 1.0": p, "temperature 0.5": p ** 2 / (p ** 2).sum(),
          "top-k 2": np.r_[p[:2], 0, 0, 0] / p[:2].sum(), "top-p 0.8": np.r_[p[:3], 0, 0] /
              p[:3].sum()}
how = {"temperature 1.0": dict(), "temperature 0.5": dict(temperature=0.5), "top-k 2":
    dict(top_k=2), "top-p 0.8": dict(top_p=0.8)}
print(f"probabilities: {p.tolist()}\n\n{'sampler':<17}{'expected':<42}{'drawn':<42}{'largest
    gap':>11}")
rng = np.random.RandomState(0)
for name in expect:
    got = np.bincount(sample_next(lg, rng, **how[name]), minlength=5) / 200000
    print(f"{name:<17}{str(np.round(expect[name], 3).tolist()):<42}{str(np.round(got,
        3).tolist()):<42}{np.abs(got - expect[name]).max():>11.4f}")
print(f"greedy: {sample_next(lg[:3], rng, temperature=0).tolist()} (always token 0)")
```

- `np.log(p)[None].repeat(200000, axis=0)` — the same scores 200,000 times: one draw per row.
- `np.bincount(..., minlength=5) / 200000` — how often each token was drawn.
- `p ** 2 / (p ** 2).sum()` — temperature 0.5 is p to the power 1/t = 2, renormalised.

REAL GRAPH



Real graph from the script: repetition, real words, and the two kinds of bits.

Quick Syntax Glossary — new or reinforced constructs

- `np.put_along_axis(a, idx, v, axis)` — write values at index positions along an axis.
- `np.cumsum(p, axis=-1)` — running totals along each row.
- `f(**settings)` — a dictionary's entries as keyword arguments.
- `d.setdefault(k, {})` — d[k], created empty the first time.
- `np.bincount(a, minlength=n)` — how often each value 0..n-1 occurs.
- `re.findall(pattern, s)` — every match of a pattern.

End of Day 93 syntax reference. For the experiments and what they showed, see the Day 93 complete guide.